# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 3 – DESIGN TASK

| | | | |
|---|---|---|---|
| Course Code: | DSA03 | Course Title: | Natural Language Processing |
| CO Assessed: | CO3 – Precision | Assessment: | Assessment Tool 3 – Design Task |
| Weightage: | 40% of CIA | | |
| CO5 Statement: | Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3) | | |
| Student Name: | Lokeshwara H | Reg. No.: | 192325120 |
| Section / Batch: | AIML | Date: | |

Code of Conduct

*I Lokeshwara H / 192325120 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Lokeshwara H

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

| Section / Topic | Focus | Experiment Questions |
|---|---|---|
| N-gram Based Next-Word Prediction for Smart Text Input | Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation<br>Next-word prediction | Q1 |
| Smoothing and Backoff Model for Speech/Text Prediction | N-gram frequency calculation<br>Unsmoothed probability estimation<br>Backoff mechanism | Q2 |
| Entropy-Based Evaluation of an English Language Model | Training and testing corpus creation<br>Unigram/bigram/trigram construction<br>Probability estimation | Q3 |
| Part-of-Speech Tagging System for Text Analysis | Text preprocessing and tokenization English/Penn Treebank tagset representation<br>Rule-based POS tagging | Q4 |

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model. The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- Word/tag frequencies
- Tag transition probabilities
- Probabilities obtained from a training corpus

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

| Criteria | Weightage (%) | Level 4 – Exemplary (4) | Level 3 – Proficient (3) | Level 2 – Developing (2) | Level 1 – Beginning (1) |
|---|---|---|---|---|---|
| Understanding of Concept | 25% | Demonstrates complete understanding of design requirements and concepts | Good understanding with minor gaps | Basic understanding with some errors | Poor understanding or incorrect concepts |
| Design / Implementation | 30% | Well-structured, efficient, and responsive design implementation | Correct implementation with minor issues | Basic implementation with inconsistencies | Incorrect or incomplete implementation |
| Use of Tools | 20% | Effective and advanced use of tools/techniques | Appropriate use of tools | Limited or inconsistent use | Incorrect or minimal use |
| Analysis & Evaluation | 15% | Insightful analysis with strong justification and optimization | Good analysis with justification | Basic analysis with limited depth | Poor or no analysis |
| Documentation / Clarity | 10% | Clear, well-structured, and properly presented solution | Mostly clear with minor issues | Basic clarity, missing details | Poor or unclear documentation |

| Q. No. / Criteria Level | Understanding of Concepts | Experimental Design | Use of Tools | Analysis of Results | Documentation | Sub. Total | Total % |
|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |
| 2 | | | | | | | |
| 3 | | | | | | | |
| 4 | | | | | | | |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

# Q1 – N-gram Based Next-Word Prediction

Aim

To implement an N-gram language model using unigram, bigram and trigram probabilities and predict the top-5 most probable next words.

Algorithm

1. Create an English corpus.
2. Convert text to lowercase.
3. Tokenize the corpus.
4. Construct unigram, bigram and trigram frequency counts.
5. Calculate Maximum Likelihood probabilities.
6. Accept an incomplete sentence.
7. Predict the next five words.
8. Demonstrate zero probability for unseen N-grams.
9. Calculate prediction accuracy using test cases.

Python Program

```
import re
from collections import Counter, defaultdict

#
# TRAINING CORPUS #

corpus = """
artificial intelligence is transforming modern technology
artificial intelligence is used in healthcare
artificial intelligence is changing the world
machine learning is a branch of artificial intelligence
machine learning is widely used in applications machine
learning can solve complex problems machine learning
can predict future outcomes
deep learning is a powerful technique
deep learning can process large amounts of data
natural language processing is a field of artificial intelligence natural
language processing can understand human language natural language
processing is used in chatbots
data science is an important technology data
science can analyze large datasets students learn
natural language processing
```

```
students learn machine learning
students use artificial intelligence
technology is changing the world
computer science is a popular field
artificial intelligence can improve healthcare
artificial intelligence can improve education """

#
# TOKENIZATION
#

def tokenize(text):
    return re.findall(r'\b[a-z]+\b', text.lower())

tokens = tokenize(corpus)

#
# N-GRAM COUNTS
#


unigram_counts = Counter(tokens)

bigram_counts = Counter(
    zip(tokens[:-1], tokens[1:])
)

trigram_counts = Counter(
    zip(tokens[:-2], tokens[1:-1], tokens[2:])
)

total_words = len(tokens) #
# PROBABILITY FUNCTIONS
#

def unigram_probability(word):
    return unigram_counts[word] / total_words


def bigram_probability(w1, w2): if
    unigram_counts[w1] == 0:
        return 0
```

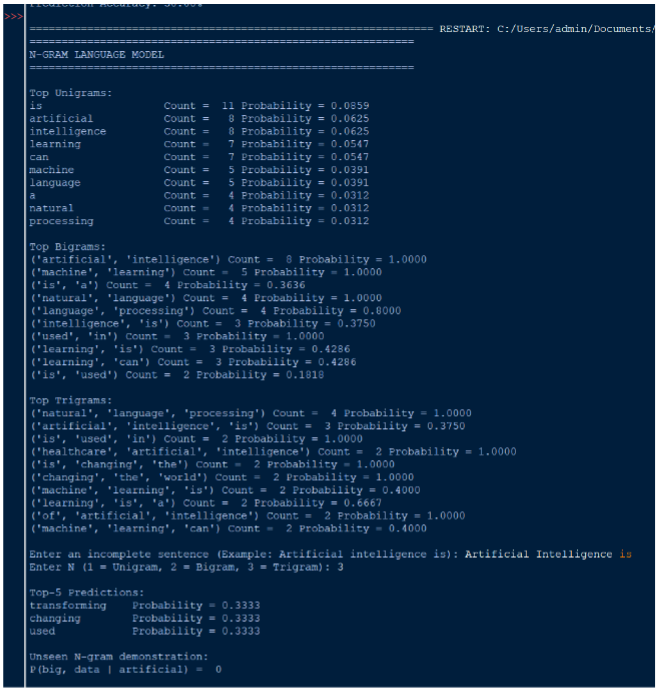
```
    return bigram_counts[(w1, w2)] / unigram_counts[w1]


def trigram_probability(w1, w2, w3):
    denominator = bigram_counts[(w1, w2)]

    if denominator == 0: return
        0

    return trigram_counts[(w1, w2, w3)] / denominator


#
# PREDICTION
#

def    predict_next(sentence,    n):
    words    =    tokenize(sentence)
    candidates = set(tokens)

    predictions = []

    for word in candidates: if

        n == 1:
            probability = unigram_probability(word)

        elif n == 2:
            if len(words) < 1:
                probability = 0
            else:
                probability = bigram_probability(words[-1], word)

        elif n == 3:
            if len(words) < 2:
                probability = 0
            else:
                probability = trigram_probability(
                    words[-2], words[-1], word
                )
        else:
            raise ValueError("N must be 1, 2, or 3")
```

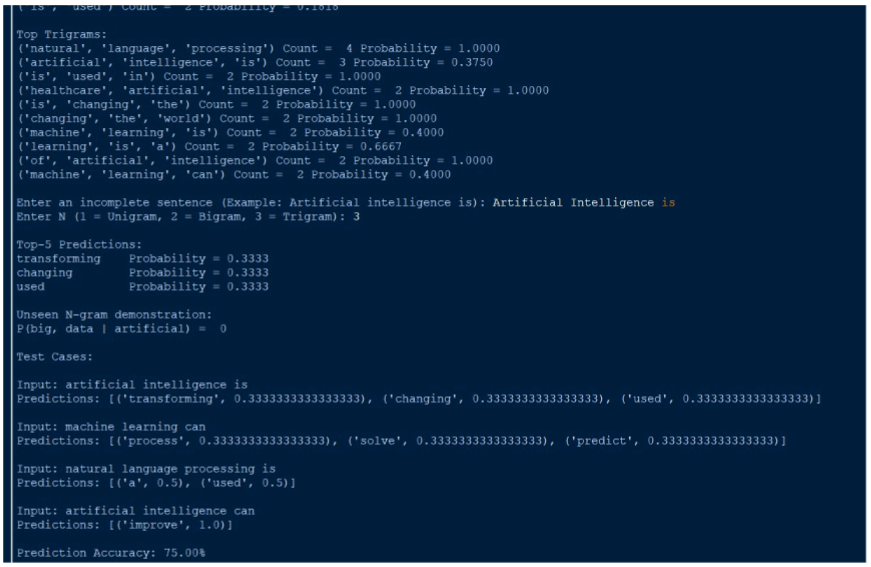
```
        if probability > 0: predictions.append((word,
            probability))

    predictions.sort(key=lambda x: x[1], reverse=True) return

    predictions[:5]


#
# DISPLAY COUNTS
#

print("=" * 60)
print("N-GRAM LANGUAGE MODEL")
print("=" * 60)

print("\nTop Unigrams:")
for word, count in unigram_counts.most_common(10):
    print(f"{word:20} Count = {count:3} "
          f"Probability = {count / total_words:.4f}")

print("\nTop Bigrams:")
for pair, count in bigram_counts.most_common(10): probability
    = count / unigram_counts[pair[0]]
    print(f"{pair} Count = {count:2} Probability = {probability:.4f}")

print("\nTop Trigrams:")
for triple, count in trigram_counts.most_common(10):  probability =
    count / bigram_counts[(triple[0], triple[1])] print(f"{triple} Count =
    {count:2} Probability = {probability:.4f}")


#
# USER INPUT
#

sentence = input(
    "\nEnter an incomplete sentence "
    "(Example: Artificial intelligence is): "
)

n = int(input("Enter N (1 = Unigram, 2 = Bigram, 3 = Trigram): "))

print("\nTop-5 Predictions:")
```

```
results = predict_next(sentence, n)

if results:
    for word, probability in results:
        print(f"{word:15} Probability = {probability:.4f}")
else:
    print("No prediction available. Probability = 0")



#
# UNSEEN N-GRAM DEMONSTRATION #


print("\nUnseen N-gram demonstration:") print(
    "P(big, data | artificial) = ",
    trigram_probability("artificial", "big", "data")
)

#
# TEST CASES #

test_cases = [
    ("artificial intelligence is", 3),
    ("machine learning can", 3),
    ("natural language processing is", 3),
    ("artificial intelligence can", 3)
]

correct = 0
total = len(test_cases)

print("\nTest Cases:")

for sentence, n in test_cases:

    prediction = predict_next(sentence, n)

    print(f"\nInput: {sentence}")
    print("Predictions:", prediction)
```

```
    if prediction:
        # Demonstration-based expected words expected =
        {
            "artificial intelligence is": "transforming",
            "machine learning can": "solve", "natural
            language processing is": "a", "artificial
            intelligence can": "improve"
        }

        if prediction[0][0] == expected[sentence]: correct
            += 1

accuracy = (correct / total) * 100

print("\nPrediction Accuracy:",
      f"{accuracy:.2f}%")
```

OUTPUT:






Result

The N-gram language model successfully calculated unigram, bigram and trigram probabilities and predicted the next words. The experiment also demonstrated that an unseen N-gram receives probability 0 in an unsmoothed model.

# Q2 – Backoff and Deleted Interpolation

Aim

To implement an enhanced language model using backoff and interpolation to overcome the zero-probability problem.

Formula

Backoff:

$$P(w|context) = \lambda_1 P(w) + \lambda_2 P(w|w_{n-1}) + \lambda_3 P(w|w_{n-2}, w_{n-1})$$

Trigram → Bigram → Unigram

Interpolation:

where:

λ1 + λ2 + λ3 = 1

Python Program

```
import re
from collections import Counter

#
# CORPUS
#

corpus = """
machine learning can solve complex problems machine
learning can predict future outcomes machine learning is
a branch of artificial intelligence machine learning is
widely used in applications artificial intelligence can
improve healthcare  artificial intelligence can improve
education
artificial intelligence is transforming technology
natural language processing can understand human language natural
language processing is used in chatbots
deep learning can process large datasets deep
learning is a powerful technique data science
can analyze large datasets data science is an
important technology students learn machine
learning
students learn artificial intelligence """

def tokenize(text):
    return re.findall(r'\b[a-z]+\b', text.lower())

tokens = tokenize(corpus)

#
# COUNTS
#

unigrams = Counter(tokens) bigrams =

Counter(
    zip(tokens[:-1], tokens[1:])
)

trigrams = Counter(
    zip(tokens[:-2], tokens[1:-1], tokens[2:])
```

```
)

total = len(tokens)


#
# PROBABILITIES
#

def unigram_prob(word): return
    unigrams[word] / total


def bigram_prob(w1, w2): if
    unigrams[w1] == 0:
        return 0

    return bigrams[(w1, w2)] / unigrams[w1]


def trigram_prob(w1, w2, w3):
    denominator = bigrams[(w1, w2)]

    if denominator == 0: return
        0

    return trigrams[(w1, w2, w3)] / denominator


#
# UNSMOOTHED MODEL #

def unsmoothed_prediction(sentence):
    words = tokenize(sentence) candidates
    = set(tokens)

    results = []

    for word in candidates: if

        len(words) >= 2:
            probability = trigram_prob(
                words[-2], words[-1], word
```

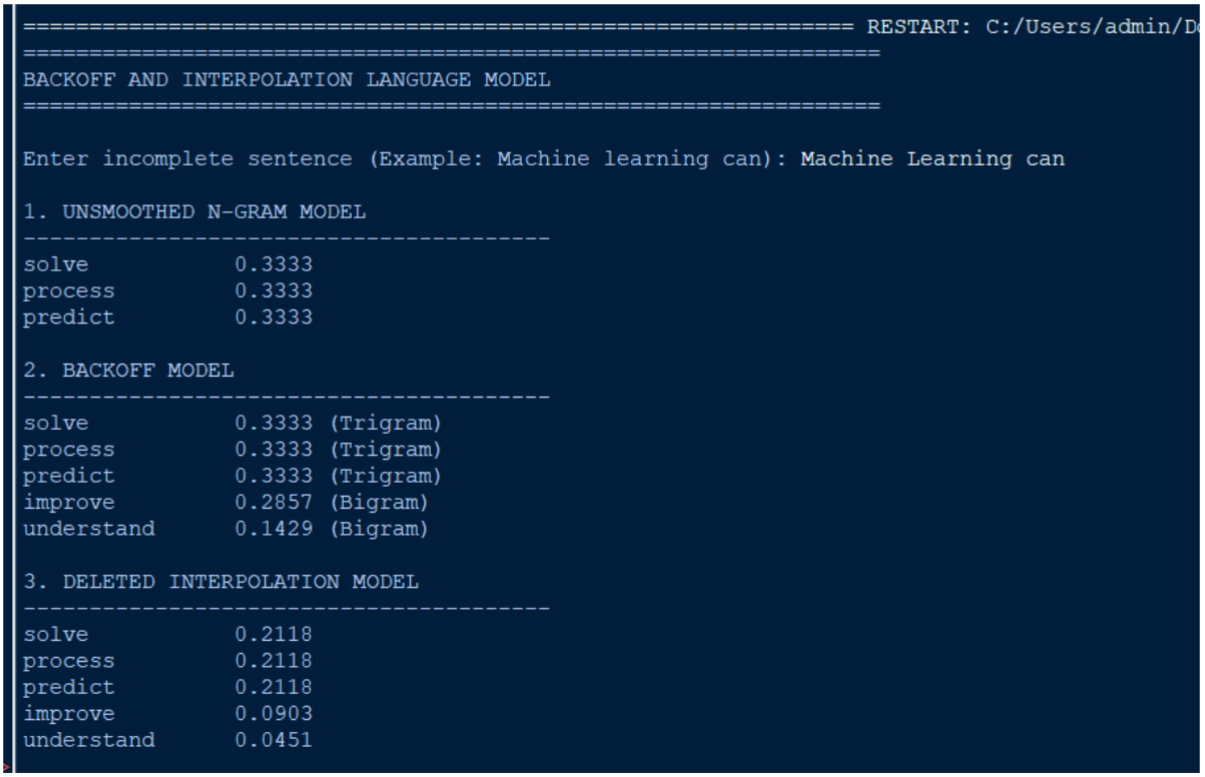
```
            )
        else:
            probability = 0

        if probability > 0: results.append((word,
            probability))

    return sorted( results,
        key=lambda x: x[1],
        reverse=True
    )[:5]


#
# BACKOFF MODEL
#

def backoff_probability(words, candidate): if

    len(words) >= 2:

        p = trigram_prob(
            words[-2],
            words[-1],
            candidate
        )

        if p > 0:
            return p, "Trigram"

    if len(words) >= 1:

        p = bigram_prob(
            words[-1],
            candidate
        )

        if p > 0:
            return p, "Bigram"

    p = unigram_prob(candidate) if

    p > 0:
```

```
        return p, "Unigram"

    return 0, "None"


  def backoff_prediction(sentence):

    words = tokenize(sentence) results =
    []

      for candidate in set(tokens):

        probability, level = backoff_probability(
            words, candidate
        )

        if probability > 0:
            results.append(
                (candidate, probability, level)
            )

    results.sort( key=lambda
        x: x[1], reverse=True
    )

    return results[:5]


#
# INTERPOLATION MODEL
#

lambda1 = 0.2
lambda2 = 0.3
lambda3 = 0.5


def interpolation_probability(words, candidate): p1

    = unigram_prob(candidate)

    p2 = 0
```

```
    p3 = 0

    if len(words) >= 1:  p2
        = bigram_prob(
            words[-1],
            candidate
        )

    if len(words) >= 2:  p3
        = trigram_prob(
            words[-2],
            words[-1],
            candidate
        )

    probability = (
        lambda1 * p1 +
        lambda2 * p2 +
        lambda3 * p3
    )

    return probability


def interpolation_prediction(sentence): words =

    tokenize(sentence)
    results = []

    for candidate in set(tokens):

        probability = interpolation_probability(
            words,
            candidate
        )

        results.append( (candidate,
            probability)
        )

    results.sort( key=lambda
        x: x[1], reverse=True
    )
```

```
    return results[:5]


#
# MAIN
#

print("=" * 65)
print("BACKOFF AND INTERPOLATION LANGUAGE MODEL")
print("=" * 65)

sentence = input(
    "\nEnter incomplete sentence " "(Example:
    Machine learning can): "
)

print("\n1. UNSMOOTHED N-GRAM MODEL")
print("-" * 40)

for word, probability in unsmoothed_prediction(sentence):
    print(
        f"{word:15} {probability:.4f}"
    )


print("\n2. BACKOFF MODEL")
print("-" * 40)

for word, probability, level in backoff_prediction(sentence):
    print(
        f"{word:15} {probability:.4f} ({level})"
    )


print("\n3. DELETED INTERPOLATION MODEL")
print("-" * 40)

for word, probability in interpolation_prediction(sentence):
    print(
        f"{word:15} {probability:.4f}"
    )
```

OUTPUT:




Result

The backoff model successfully used lower-order N-grams when a higher-order N-gram was unavailable. The interpolation model combined unigram, bigram and trigram probabilities, providing better coverage for sparse data.

# Q3 – Entropy-Based Evaluation

Aim

To calculate and compare the entropy of unigram, bigram, trigram and smoothed language models.

Formula

$$H(W) = -\frac{1}{N}\sum_{i=1}^{N}\log_2 P(w_i|context)$$

)

Lower entropy → more predictable

Higher entropy → less predictable

Python Program

```
import re
import math
from collections import Counter

#
# TRAINING CORPUS #

training_text = """
the cat sits on the mat the
cat eats food
the cat likes milk
the dog sits on the mat the
dog eats food
the dog likes milk
the boy plays in the park the
girl plays in the park
students learn natural language processing
students learn machine learning
machine learning is useful
artificial intelligence is useful
natural language processing is useful """

#
# TESTING CORPUS #

test_sentences = [
    "the cat sits on the mat",
    "the quantum processor redesigned the"
]
```

```
def tokenize(text):
    return re.findall(r'\b[a-z]+\b', text.lower())


train_tokens = tokenize(training_text)

unigrams = Counter(train_tokens) bigrams =

Counter(
    zip(train_tokens[:-1], train_tokens[1:])
)

trigrams = Counter(
    zip(
        train_tokens[:-2],
        train_tokens[1:-1],
        train_tokens[2:]
    )
)

total_words = len(train_tokens)
vocabulary = set(train_tokens) V
= len(vocabulary)


#
# PROBABILITY FUNCTIONS
#

def unigram_probability(word):

    return unigrams[word] / total_words


def bigram_probability(w1, w2): if

    unigrams[w1] == 0:
        return 0

    return bigrams[(w1, w2)] / unigrams[w1]


def trigram_probability(w1, w2, w3):
```

```
    denominator = bigrams[(w1, w2)]

    if denominator == 0: return
        0

    return trigrams[(w1, w2, w3)] / denominator


#
# ADD-ONE SMOOTHING #

def smoothed_unigram_probability(word):

    return (unigrams[word] + 1) / (
        total_words + V
    )


def smoothed_bigram_probability(w1, w2):

    return (bigrams[(w1, w2)] + 1) / (
        unigrams[w1] + V
    )


def smoothed_trigram_probability(w1, w2, w3): return

    (trigrams[(w1, w2, w3)] + 1) / (
        bigrams[(w1, w2)] + V
    )


#
# ENTROPY
#

def calculate_entropy(sentence, model): words =

    tokenize(sentence) log_probability = 0
    count = 0
```

```
for i, word in enumerate(words): if

    model == "unigram":

        probability = unigram_probability(word)

    elif model == "bigram":

        if i == 0:
            probability = unigram_probability(word) else:
            probability = bigram_probability(
                words[i - 1],
                word
            )

    elif model == "trigram":

        if i < 2:
            probability = unigram_probability(word)

        else:
            probability = trigram_probability(
                words[i - 2],
                words[i - 1],
                word
            )

    elif model == "smoothed": if

        i < 2:
            probability = smoothed_unigram_probability( word
            )

        else:
            probability = smoothed_trigram_probability( words[i
                - 2],
                words[i - 1],
                word
            )

    if probability <= 0:
```

```
            return float("inf")

        log_probability += math.log2(probability)
        count += 1

    return -log_probability / count


#
# MAIN
#

print("=" * 70)
print("ENTROPY-BASED LANGUAGE MODEL EVALUATION")
print("=" * 70)

for sentence in test_sentences:

    print("\nSentence:")
    print(sentence)


    print("\nEntropy Results:")

    for model in [
        "unigram",
        "bigram",
        "trigram",
        "smoothed"
    ]:

        entropy = calculate_entropy(
            sentence,
            model
        )

        if entropy == float("inf"):
            print(
                f"{model.capitalize():12}: Undefined " "(zero
                probability)"
            )
        else:
            print(
                f"{model.capitalize():12}: "
                f"{entropy:.4f} bits"
```

```
        )

    print("\nClassification:")

    entropy = calculate_entropy(
        sentence,
        "smoothed"
    )

    if entropy < 3:
        print("Low predictive uncertainty") else:
        print("High predictive uncertainty")
```

OUTPUT:

```
>>>
    ========== RESTART: C:/Users/admin/Docu
    ========================================
    ENTROPY-BASED LANGUAGE MODEL EVALUATION
    ========================================

    Sentence:
    the cat sits on the mat

    Entropy Results:
    Unigram     : 3.9950 bits
    Bigram      : 1.4232 bits
    Trigram     : 1.3872 bits
    Smoothed    : 3.5460 bits

    Classification:
    High predictive uncertainty

    Sentence:
    the quantum processor redesigned the

    Entropy Results:
    Unigram     : Undefined (zero probability)
    Bigram      : Undefined (zero probability)
    Trigram     : Undefined (zero probability)
    Smoothed    : 4.6639 bits

    Classification:
    High predictive uncertainty
>>>
```

Result

The experiment shows that familiar sentences have lower entropy because their words are more predictable from the training corpus. Unfamiliar sentences have higher entropy because the model is uncertain about the next words. Smoothing prevents the model from producing undefined entropy due to zero probabilities.

# Q4 – Comparative POS Tagging System

Aim

To implement and compare rule-based, stochastic and transformation-based POS tagging methods.

POS Tags

| Tag | Meaning |
|---|---|
| NN | Noun |
| VB | Verb |
| JJ | Adjective |
| RB | Adverb |
| DT | Determiner |
| PRP | Pronoun |
| IN | Preposition |
| CC | Conjunction |

```
import re
from collections import Counter, defaultdict

#
# TRAINING SENTENCES #

training_data = [
    ("I book a ticket", ["PRP", "VB", "DT", "NN"]),
    ("I read a book", ["PRP", "VB", "DT", "NN"]),
    ("The book is interesting", ["DT", "NN", "VB", "JJ"]),
    ("She reads books", ["PRP", "VB", "NNS"]),
    ("The boy plays football", ["DT", "NN", "VB", "NN"]),
    ("The girl likes music", ["DT", "NN", "VB", "NN"]),
    ("He eats good food", ["PRP", "VB", "JJ", "NN"]),
```

```
    ("They play in the park",
     ["PRP", "VB", "IN", "DT", "NN"]),
    ("The dog runs quickly",
     ["DT", "NN", "VB", "RB"]),
    ("She writes a letter", ["PRP",
     "VB", "DT", "NN"])
]


#
# RULE-BASED TAGGER
#

lexicon = {
    "i": "PRP",
    "he": "PRP",
    "she": "PRP",
    "they": "PRP",
    "we": "PRP",

    "the": "DT",
    "a": "DT",
    "an": "DT",

    "is": "VB",
    "am": "VB",
    "are": "VB",
    "book": "NN",
    "read": "VB",
    "reads": "VB",
    "plays": "VB",
    "play": "VB",
    "likes": "VB",
    "eats": "VB",
    "writes": "VB",

    "in": "IN",
    "on": "IN",

    "good": "JJ",
    "interesting": "JJ",

    "quickly": "RB",
```

```
    "ticket": "NN",
    "music": "NN",
    "football": "NN",
    "food": "NN",
    "letter": "NN",
    "park": "NN",
    "dog": "NN",
    "boy": "NN",
    "girl": "NN"
}


def rule_based_tag(sentence):

    words = sentence.lower().split() tags
    = []

    for word in words:

        if word in lexicon:
            tag = lexicon[word]

        elif word.endswith("ly"): tag
            = "RB"

        elif word.endswith("ing"):
            tag = "VBG"

        elif word.endswith("ed"):
            tag = "VBD"

        elif word.endswith("s"):
            tag = "NNS"

        elif word[0].isupper():
            tag = "NNP"

        else:
            tag = "NN"

        tags.append((word, tag))

    return tags
```

```
#
# STOCHASTIC TAGGER #

word_tag_counts = defaultdict(Counter)
tag_transition_counts = defaultdict(Counter) tag_counts
= Counter()

for sentence, tags in training_data:

    words = sentence.lower().split() for

    word, tag in zip(words, tags):

        word_tag_counts[word][tag] += 1
        tag_counts[tag] += 1

    for t1, t2 in zip(tags[:-1], tags[1:]):
        tag_transition_counts[t1][t2] += 1


def stochastic_tag(sentence): words =

    sentence.lower().split() result = []

    previous_tag = None

    for word in words:

        if word in word_tag_counts:

            candidates = word_tag_counts[word] best_tag =

            None
            best_score = -1

            for tag, count in candidates.items(): word_probability =

                (
                    count / sum(candidates.values())
                )
```

```
            if previous_tag is not None:

                transition_probability = ( tag_transition_counts[
                        previous_tag
                    ][tag]
                    /
                    max(
                        1,
                        sum(
                            tag_transition_counts[ previous_tag
                            ].values()
                        )
                    )
                )

            else:
                transition_probability = 1

            score = ( word_probability
                * transition_probability
            )

            if  score  >  best_score:
                best_score  =  score
                best_tag = tag


        tag  =  best_tag

    else:
        tag = "NN"

    result.append((word, tag))
    previous_tag = tag

return result


#
# TRANSFORMATION-BASED TAGGER #
```

```
def transformation_based_tag(sentence):

    # Initial tagging
    result = rule_based_tag(sentence) corrected =

    []

    for i, (word, tag) in enumerate(result): #

        Transformation rules

        if word == "book":

            # "book" after "I" is a verb
            if i > 0 and result[i - 1][0] == "i":
                tag = "VB"

            # "book" after determiner is a noun
            elif i > 0 and result[i - 1][1] == "DT":
                tag = "NN"

        if word == "read":

            if i > 0 and result[i - 1][0] == "i":
                tag = "VB"

        if word.endswith("ly"):
            tag = "RB"

        corrected.append((word, tag))

    return corrected


#
# COMPARISON #

sentence = input(
    "Enter an English sentence "
    "(Example: I book a ticket): "
)
```

```
print("\n" + "=" * 60)
print("RULE-BASED POS TAGGING")
print("=" * 60)

for word, tag in rule_based_tag(sentence):
    print(f"{word:15} -> {tag}")


print("\n" + "=" * 60)
print("STOCHASTIC POS TAGGING")
print("=" * 60)

for word, tag in stochastic_tag(sentence):
    print(f"{word:15} -> {tag}")


print("\n" + "=" * 60)
print("TRANSFORMATION-BASED POS TAGGING")
print("=" * 60)

for word, tag in transformation_based_tag(sentence):
    print(f"{word:15} -> {tag}")


#
# DEMONSTRATION OF CONTEXT #

print("\n" + "=" * 60)
print("CONTEXT-SENSITIVE WORD: BOOK")
print("=" * 60)

examples = [
    "I book a ticket",
    "I read a book"
]

for example in examples: print("\nSentence:",

    example)

    print("Transformation-based:") print(
        transformation_based_tag(example)
```

)

OUTPUT:

```
========== RESTART: C:/Users/admin/D
Enter an English sentence (Example: I book a ticket): I book a ticket

============================================================
RULE-BASED POS TAGGING
============================================================
i               -> PRP
book            -> NN
a               -> DT
ticket          -> NN

============================================================
STOCHASTIC POS TAGGING
============================================================
i               -> PRP
book            -> VB
a               -> DT
ticket          -> NN

============================================================
TRANSFORMATION-BASED POS TAGGING
============================================================
i               -> PRP
book            -> VB
a               -> DT
ticket          -> NN

============================================================
CONTEXT-SENSITIVE WORD: BOOK
============================================================

Sentence: I book a ticket
Transformation-based:
[('i', 'PRP'), ('book', 'VB'), ('a', 'DT'), ('ticket', 'NN')]

Sentence: I read a book
Transformation-based:
[('i', 'PRP'), ('read', 'VB'), ('a', 'DT'), ('book', 'NN')]
```

**Accuracy Calculation**

For a POS tagger:

$$Accuracy = \frac{\text{Number of correctly tagged words}}{\text{Total number of words}} \times 100$$

Example:

Result

The three POS tagging approaches successfully assigned grammatical tags to English sentences. The experiment demonstrated that the same word can receive different tags depending on context. For example, "book" is a verb in *"I book a ticket"* and a noun in *"I read a book."*